

Generative AI & GANs

Generative AI & GANs — Concept Notes

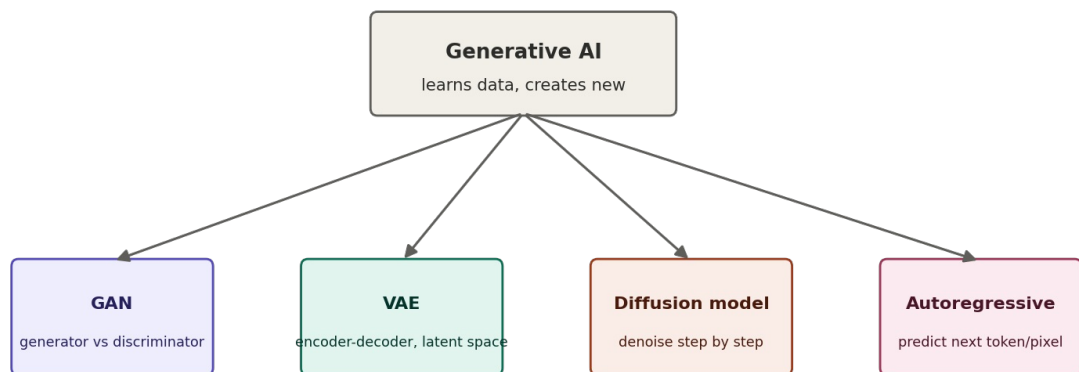
1. Generative AI — Overview

- **Discriminative models** (jo tune ab tak seekha — ANN, CNN, RNN, NLP, Transformer, LLM) sirf **classify / judge** karte hain
 - Example: “yeh image cat hai ya dog?”
- **Generative models** data ki **underlying distribution** seekhte hain aur usse **naye samples create** karte hain
 - Example: “mujhe ek naya cat image bana ke do”
- **Analogy**: discriminative painter sirf bata sakta hai “yeh Picasso hai ya nahi”; generative painter khud Picasso jaisi **nayi painting bana sakta hai**
- **Kyu zaroori hai**: naya art/image banana, missing data fill karna, data augmentation, deepfakes, voice synthesis, molecule design

Four main approaches

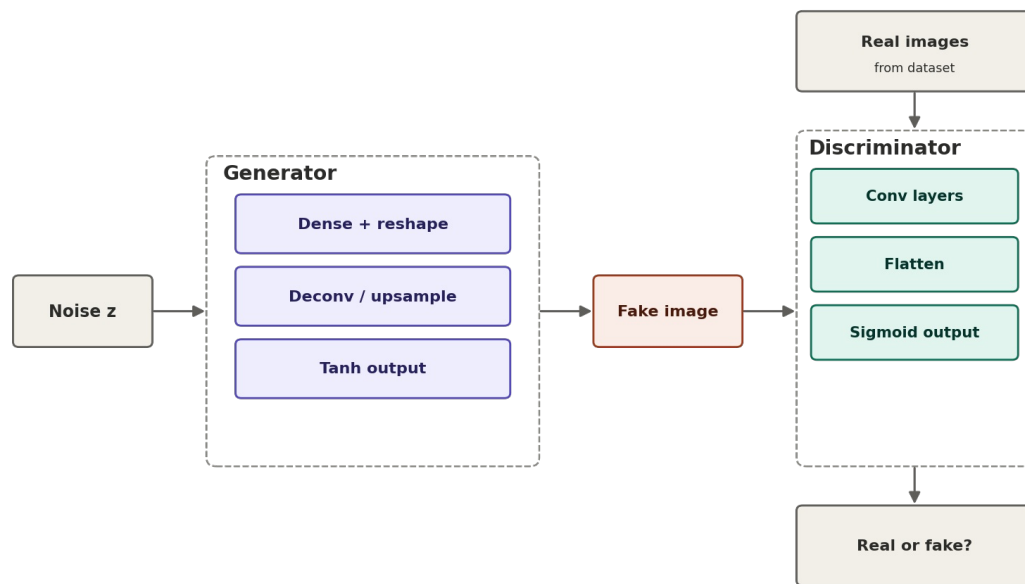
Approach	Kaise seekhta hai
GAN	Do networks compete karte hain (generator vs discriminator)
VAE	Data ko chhote latent space mein compress karke wahan se sample karta hai
Diffusion model	Random noise se shuru, step-by-step denoise karke clean data banata hai
Autoregressive (LLM type)	Ek time pe ek token/pixel predict karta hai

- Sabka goal same: **naya, realistic sample generate karna** — bas method alag hai



2. GAN — Full Architecture

- **Analogy:** ek nakli note banane wala (Generator) aur ek police wala (Discriminator) — dono ek doosre se compete karte hue train hote hain
- Generator try karta hai asli-jaisa fake data banane ka
- Discriminator try karta hai asli aur nakli mein farak pakadne ka
- Time ke saath dono improve karte hain — Generator itna acha ho jaata hai ki Discriminator bhi confuse ho jaata hai



GAN full architecture

Parts explained

- **Noise z** — random numbers ka vector, generator ka starting seed
- **Generator**
 - Dense + reshape — noise ko chhoti image-jaisi shape mein convert karta hai
 - Deconv / upsample — dheere-dheere size badhata hai (chhoti se badi image)
 - Tanh output — pixel values ko fixed range mein le aata hai, fake image ban jaati hai
- **Fake image** — Generator ka output, real world mein kabhi nahi thi
- **Real images (dataset)** — asli data, seedha dataset se Discriminator ko diya jaata hai
- **Discriminator**
 - Conv layers — image se features nikaalta hai (edges, patterns, textures)
 - Flatten — features ko ek lambi list mein convert karta hai
 - Sigmoid output — 0 se 1 ke beech ek number: 1 = real lagta hai, 0 = fake lagta hai
- **Real or fake?** — final verdict, discriminator ka confidence score

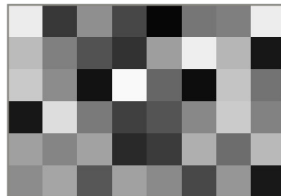
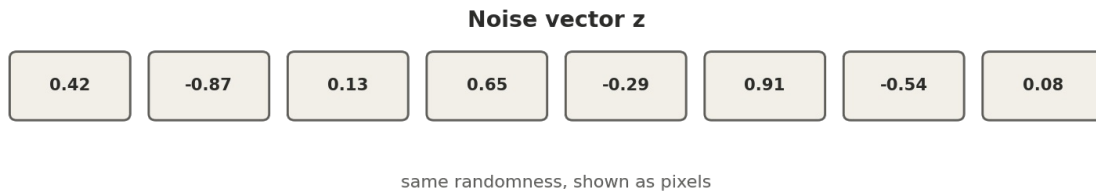
Poora cycle

1. Random noise z → Generator ko diya
2. Generator usse fake image mein convert karta hai
3. Fake image + Real image, dono Discriminator ko diye jaate hain

- Discriminator dono ko check karta hai, real ya fake decide karta hai
- Process repeat hota hai — Generator behtar fake images banata hai, Discriminator behtar pakadta hai

3. Noise Vector z

- Ek **random numbers ka vector** (jaise 100 random numbers) — Generator ka seed
- Har baar different random z se **different image** banti hai
- Shuru mein ismein koi meaning nahi hota — bas random noise
- Neeche same randomness ko pixels ke form mein dikhaya hai — jaise TV static

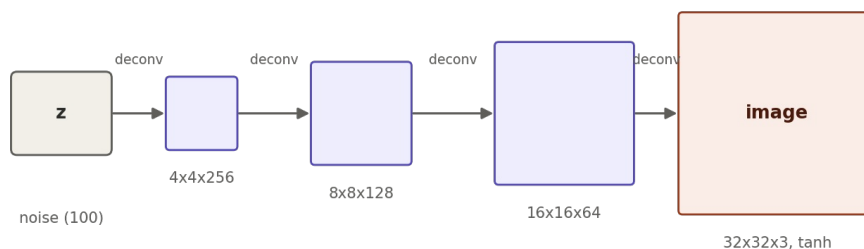


Noise vector visualization

4. Generator — Internal Flow (deep dive)

- Generator noise vector ko **dheere-dheere bada karta hai**, spatial size step-by-step badhti jaati hai
- $4 \times 4 \rightarrow 8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32$ — har deconv/upsample step pe details add hoti jaati hain (dhundhla se sharp)
- Last step: **tanh activation** se pixel values fixed range mein aate hain, final image ban jaati hai

Generator: noise to image



Generator internal flow

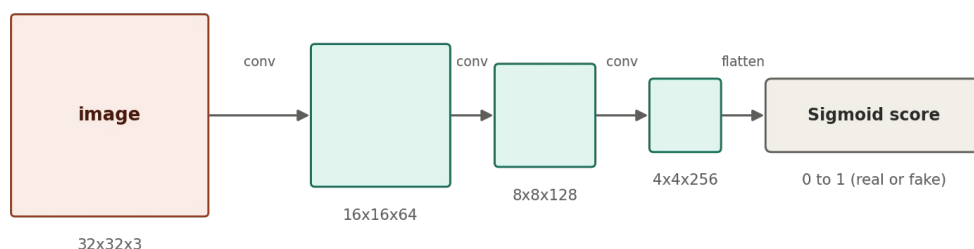
Key points

- **Dense + reshape:** random vector ko ek chhoti feature-map shape mein arrange karta hai (jaise 100 numbers $\rightarrow$ 4×4 grid)
- **Deconv / upsample layers:** spatial resolution badhaate hain, saath saath channels (depth) kam karte jaate hain
- **Tanh output:** final activation jo output ko image-jaisa banata hai

5. Discriminator — Internal Flow (deep dive)

- Discriminator ek **classifier hai** — bilkul CNN jaisa jo pehle seekha tha
- Kaam: image dekh ke decide karna “yeh asli hai ya nakli?”
- Do tarah ki images leta hai — Generator se aayi fake images, aur dataset se aayi real images
- Har baar output ek hi number deta hai: **0 se 1 ke beech**

Discriminator: image to score



Discriminator internal flow

Parts explained

- **Image input (32×32×3)** — starting point, real ya fake image (discriminator ko pata nahi hota shuru mein)
- **Conv layers (downsampling)** — $32 \times 32 \times 3 \rightarrow 16 \times 16 \times 64 \rightarrow 8 \times 8 \times 128 \rightarrow 4 \times 4 \times 256$
 - Har conv layer image pe chhota filter slide karta hai, important features nikaalta hai (edges, patterns, textures)
 - Spatial size **chhoti** hoti hai ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4$), lekin depth (channels) **badhti** hai ($3 \rightarrow 64 \rightarrow 128 \rightarrow 256$)
 - Yeh Generator ka **ulta** hai — Generator upsample karta hai, Discriminator downsample karta hai
- **Flatten** — 3D block ($4 \times 4 \times 256$) ko ek seedhi, lambi 1D list mein convert karta hai, taaki Dense layer process kar sake
- **Sigmoid score** — final Dense + Sigmoid layer, ek hi number deta hai (0-1)
 - 1 ke close $\rightarrow$ “real lagta hai”
 - 0 ke close $\rightarrow$ “fake lagta hai”

Poora flow

1. Image aati hai (real ya fake)
2. Conv layers chhota karte jaate hain, features nikaalte jaate hain
3. Flatten sab kuch ek list mein convert karta hai
4. Sigmoid ek final confidence score deta hai

Kyu important hai yeh design

- Discriminator ka structure CNN classifier jaisa hi hai kyunki kaam bhi wahi hai — image classify karna (bas 2 classes: real/fake)
- Generator aur Discriminator ek doosre ke **mirror image** hain:
 - Generator: chhota → bada (creates)
 - Discriminator: bada → chhota (judges)

Math Essentials (loss functions)

- **Discriminator loss** — real ko real, fake ko fake bolna dono chahiye:

$$L_D = -[\log D(x) + \log(1 - D(G(z)))]$$

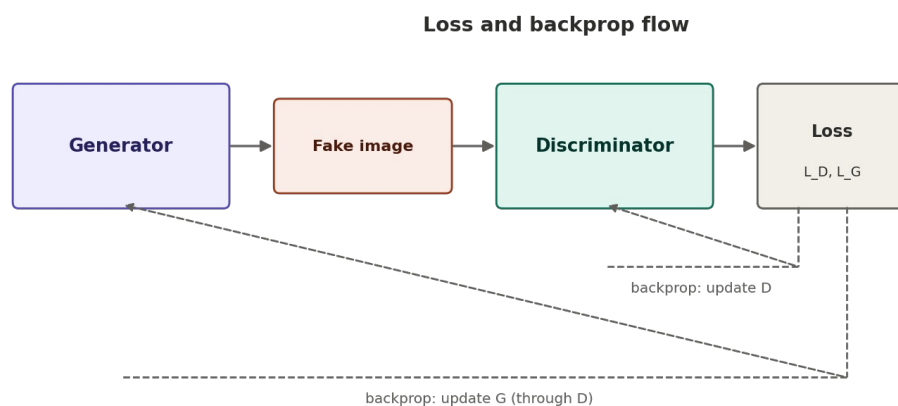
- **Generator loss** (non-saturating version):

$$L_G = -\log D(G(z))$$

- Discriminator chahta hai $D(x)$ bada ho, $D(G(z))$ chhota ho
- Generator chahta hai $D(G(z))$ bada ho (discriminator ko fool karna)

6. Loss & Backprop (detailed)

- **Loss** ek number hai jo batata hai “kitni galti ho rahi hai” — kitna door hai target se
- **Backprop (backpropagation)** woh process hai jo is galti ko **wapas, layer by layer**, network ke shuru tak bhejta hai, taaki har layer ke weights thode sudhar sakein
- GAN mein **do networks, do alag losses** hote hain kyunki dono ke goals opposite hain — isko **minimax game** kehte hain (ek jeetna chahta hai jo doosra harna chahta hai)



Loss and backprop flow

6.1 Discriminator loss — naam aur kiske liye

- **Naam:** Binary cross-entropy loss (do classes ke beech — real vs fake)
- **Kiske liye:** sirf **Discriminator** ke weights update karne ke liye
- **Kya chahta hai:** real image pe output 1 ke close, fake image pe output 0 ke close

$$L_D = -[\log D(x) + \log(1 - D(G(z)))]$$

- Symbols: $D(x)$ = discriminator ka score real image pe, $D(G(z))$ = discriminator ka score fake image pe

6.2 Generator loss — naam aur kiske liye

- **Naam:** Non-saturating generator loss (yeh version practically zyada use hoti hai kyunki gradient better milta hai)
- **Kiske liye:** sirf **Generator** ke weights update karne ke liye
- **Kya chahta hai:** discriminator uski fake image ko real samjhe (output 1 ke close ho)

$$L_G = -\log D(G(z))$$

- **Important:** formula mein hamesha $G(z)$ hi aata hai, $G(x)$ nahi — Generator sirf noise z leta hai input mein, kabhi real image x nahi dekhta. x sirf Discriminator ko directly milta hai.

6.3 Poora setup (values jo hum use karenge)

Symbol	Value	Kya hai
z	0.5	noise input
W_g, b_g	0.8, 0.1	Generator ke weight, bias
W_d, b_d	1.2, -0.3	Discriminator ke weight, bias
learning rate (η)	0.1	kitna bada step lena hai update mein

6.4 Forward pass — pehle aage se guzaro

- **Generator (tanh layer):**

$$z_g = W_g \times z + b_g = 0.8 \times 0.5 + 0.1 = 0.5$$

$$G(z) = \tanh(z_g) = \tanh(0.5) = 0.4621 \leftarrow \text{yeh fake image hai}$$

- **Discriminator (sigmoid layer), fake image pe:**

$$z_d = W_d \times G(z) + b_d = 1.2 \times 0.4621 - 0.3 = 0.2545$$

$$D(G(z)) = \text{sigmoid}(0.2545) = 0.5633 \leftarrow \text{yeh discriminator ka score hai}$$

- **Generator loss:**

$$L_G = -\log(0.5633) = 0.5744$$

- Is number (0.5744) se hume sirf pata chalta hai “kitni galti hai” — weight update ke liye hume iska **gradient** chahiye, ab woh nikalte hain, layer by layer wapas jaate hue.

6.5 Backprop — layer by layer wapas jaana (chain rule)

Poora path: **Loss → Sigmoid (Discriminator) → Discriminator weight → Tanh (Generator) → Generator weight**

Chain rule formula:

$$\partial L_G / \partial W_g = (\partial L_G / \partial D) \times (\partial D / \partial z_d) \times (\partial z_d / \partial G(z)) \times (\partial G(z) / \partial z_g) \times (\partial z_g / \partial W_g)$$

Layer 1 — Loss se (log function ka derivative):

- Formula: $\partial L_G / \partial D(G(z)) = -1/D(G(z))$
- Value: $-1/0.5633 = -1.7753$

Layer 2 — Sigmoid layer se (Discriminator ke andar):

- Formula: $\partial D/\partial z_d = D \times (1 - D)$
- Value: $0.5633 \times (1 - 0.5633) = 0.5633 \times 0.4367 = \mathbf{0.2460}$

Layer 3 — Discriminator ka weight se (kitna G(z) discriminator ke andar asar karta hai):

- Formula: $\partial z_d/\partial G(z) = W_d$
- Value: **1.2** (seedha weight hi hai)

Layer 4 — Tanh layer se (Generator ke andar):

- Formula: $\partial G(z)/\partial z_g = 1 - G(z)^2$
- Value: $1 - (0.4621)^2 = 1 - 0.2135 = \mathbf{0.7865}$

Layer 5 — Generator ka weight se (input z hi gradient hai):

- Formula: $\partial z_g/\partial W_g = z$
- Value: **0.5**

6.6 Sab layers ko multiply karo (chain rule jodna)

Pehle beech ka combined gradient nikaalo (Loss se Generator output tak):

$$\partial L_G/\partial z_g = (-1.7753) \times (0.2460) \times (1.2) \times (0.7865)$$

$$\text{Step by step: } -1.7753 \times 0.2460 = -0.4367 - 0.4367 \times 1.2 = -0.5240 - 0.5240 \times 0.7865 = \mathbf{-0.4122}$$

Ab isse Generator ke weight aur bias tak le jao:

$$\partial L_G/\partial W_g = -0.4122 \times 0.5 = \mathbf{-0.2061}$$

$$\partial L_G/\partial b_g = -0.4122 \times 1 = \mathbf{-0.4122}$$

6.7 Weight update (gradient descent)

$$W_{g_new} = W_g - \eta \times (\partial L_G/\partial W_g) = 0.8 - 0.1 \times (-0.2061) = \mathbf{0.8206}$$

$$b_{g_new} = b_g - \eta \times (\partial L_G/\partial b_g) = 0.1 - 0.1 \times (-0.4122) = \mathbf{0.1412}$$

- Gradient **negative** aaya, isliye weight **badha** — gradient descent hamesha gradient ke **opposite direction** mein move karta hai
- Yeh update sirf Generator ke weights ke liye hai — Discriminator ka apna alag update hota hai L_D se (seedha, bina kisi extra chain ke, kyunki Discriminator khud last layer hai)

6.8 Summary — kaunsa gradient kahan se aaya

Step	Kis layer se	Formula	Value
1	Loss (log)	$-1/D(G(z))$	-1.7753
2	Sigmoid (Discriminator)	$D(1-D)$	0.2460
3	Discriminator weight	W_d	1.2
4	Tanh (Generator)	$1-G(z)^2$	0.7865
5	Generator weight input	z	0.5
Final	—	sab multiply	-0.4122 (phir $\times z = -0.2061$ for W_g)

- Yehi tareeka hai jisse gradient **Discriminator ke andar se hoke** (sigmoid + weight) Generator tak (tanh + weight) pahunchta hai — isiliye diagram mein Generator ka backprop arrow Discriminator ke through wapas jaata hai, seedha Loss se nahi

Common Pitfalls (to remember later)

- Generator aur Discriminator ka balance maintain karna mushkil hai — ek zyada strong ho jaaye toh training unstable ho jaati hai
- Mode collapse — generator sirf limited variety ke fake samples banane lagta hai